

Bird’s-Eye-View Confidence Mapping for Autonomous Driving

Dev Jayram
Stanford University
devj10@stanford.edu

Marissa Liu
Stanford University
mliu4770@stanford.edu

Hannah Shu
Stanford University
hshu100@stanford.edu

Abstract

Autonomous driving planners need spatially grounded uncertainty, not just detection confidence. We present an uncertainty-aware perception pipeline combining Monte Carlo DropBlock inference with YOLOv8n to estimate per-detection uncertainty and project it into a Bird’s-Eye View (BEV) confidence map via LiDAR-assisted depth back-projection. A five-component score aggregates variance in box position, size, confidence, and class predictions across $T=20$ stochastic passes, splatted onto a 200×200 BEV grid with Gaussian kernels. Fine-tuned on nuScenes v1.0, our detector achieves 0.580 mAP@50 (car: 0.854). A single MC-DropBlock pass yields 0.468 mAP@50; fusing 20 passes recovers 0.511. A Kolmogorov–Smirnov test confirms statistically distinct confidence distributions across passes ($p < 0.01$). LiDAR lifting accurately places detections at close and mid range, degrading beyond 30 m. Fusing all six surround cameras produces a coherent ego-frame confidence field, rendered as a BEV heatmap video. Our results show that structured stochastic inference yields uncertainty estimates that are both statistically valid and spatially interpretable for downstream planning.

1. Introduction

Perception is the first step in an autonomous driving pipeline, but it is also one of the places where important information can be lost. Modern object detectors such as YOLOv8n [1] and DETR [2] output bounding boxes, class labels, and confidence scores, but these scores are not the same as calibrated measures of uncertainty [7]. A detector may assign a high confidence score to both a clearly visible sedan and a partially occluded cyclist in poor weather, even though the second prediction should clearly be treated with more caution. For a downstream planner, this distinction matters: acting on an overconfident but incorrect detection can lead to unsafe braking, missed obstacles, or poor trajectory choices. This makes uncertainty a core part of reliable perception rather than a secondary diagnostic.

A useful uncertainty estimate should also be expressed

in the coordinate system where driving decisions are made. In autonomous driving, many planning, occupancy mapping, and risk-assessment modules operate in Bird’s-Eye View (BEV), a top-down grid aligned with the ego vehicle. Image-space uncertainty is informative, but it is less directly actionable because planners reason about physical locations in front of and around the vehicle. By projecting detection uncertainty into BEV, the system can represent not only where an object is likely to be, but also how reliable that estimate is in real-world space. This allows uncertain regions to be treated more cautiously, even when the detector still produces a valid bounding box.

This paper presents an uncertainty-aware perception pipeline that estimates detection uncertainty with Monte Carlo DropBlock and projects that uncertainty into a BEV confidence map. Rather than learning a full image-to-BEV representation from scratch, we use the synchronized LiDAR sweeps in the full nuScenes v1.0 dataset to back-project 2D detections into ego-frame coordinates. This lets us focus on the central question of the project: whether uncertainty estimated in image space can be carried into a spatial representation that is more useful for downstream autonomous-driving planning.

This paper makes three contributions:

- MC-DropBlock uncertainty estimation.** We use DropBlock [4] during Monte Carlo inference to introduce structured variation into YOLOv8n’s convolutional features while keeping batch-normalization statistics fixed. For each frame, we run $T=20$ stochastic forward passes and aggregate variation in box position, box size, confidence, and class predictions into a five-component uncertainty score for each detected object.
- LiDAR-assisted BEV lifting.** We lift 2D detections into 3D ego space using synchronized LiDAR sweeps from the full nuScenes v1.0 dataset. For each camera image, we project LiDAR points into image space and estimate each object’s depth from the points inside its detection box. This produces BEV (x, y) positions with an average localization error of 0.43m against nuScenes ground-truth annotations, with reliable placement at

close and mid range and degradation beyond 30m.

3. **Spatial confidence maps.** We project each detection’s uncertainty onto a 200×200 BEV grid spanning $[0, 50]$, $m \times [-25, 25]$, m using Gaussian splatting. Fusing detections from all six surround cameras yields a coherent top-down confidence field around the ego vehicle, which we render as a frame-by-frame BEV heatmap video over a full nuScenes scene.¹

We evaluate our pipeline on the full nuScenes v1.0 dataset [8], which provides synchronized surround-view cameras, LiDAR sweeps, calibration matrices, and ground-truth 3D annotations. Our fine-tuned YOLOv8n detector achieves 0.580 mAP@50 and 0.327 mAP@50–95 on the validation split, with strong per-class performance on frequent objects (car: 0.854 mAP@50). Under stochastic inference, a single MC-DropBlock pass yields 0.468 mAP@50; fusing 20 passes recovers 0.511. A Kolmogorov–Smirnov test confirms that paired stochastic passes produce statistically distinct confidence distributions ($p < 0.01$), validating that MC-DropBlock captures meaningful predictive uncertainty rather than noise.

2. Related Work

2.1. Uncertainty Estimation in Neural Networks

A foundational approach to uncertainty estimation in deep learning is Monte Carlo Dropout. Gal and Ghahramani [3] showed that keeping Dropout active at test time can be interpreted as approximate Bayesian inference in a deep Gaussian process. In practice, this gives a simple way to estimate model uncertainty: the same input is passed through the network multiple times with different Dropout masks, and the variation across predictions is used as a proxy for uncertainty. This idea is useful for perception because it gives more information than a single confidence score. However, standard Dropout is not especially well matched to convolutional feature maps. Since neighboring activations in convolutional layers are spatially correlated, dropping isolated activations often leaves enough nearby information for the model to make nearly the same prediction.

Ghiassi et al. [4] addressed this limitation by introducing DropBlock, a structured form of Dropout that removes contiguous regions of a feature map rather than individual activations. This makes the stochastic perturbation more meaningful for dense vision models, where spatial structure matters. Yelleni et al. [5] later applied this idea in a Monte Carlo setting for object detection, showing that MC-DropBlock can improve uncertainty modeling and calibration in YOLO-based detection architectures. Our work is similar in that we

also use MC-DropBlock to estimate detection uncertainty. However, instead of stopping at image-space uncertainty, we carry the resulting uncertainty into BEV so that it can be used in the coordinate frame relevant to downstream driving decisions.

Zhao et al. [6] applied Monte Carlo Dropout to YOLOv5 for uncertainty estimation in real-time autonomous-driving object detection. Their work shows that stochastic inference can be used to estimate both classification and localization uncertainty while preserving practical detector speed. Our work follows the same general direction, but differs in three ways: we use the newer YOLOv8 backbone, replace standard neuron-level Dropout with spatially structured DropBlock, and explicitly project uncertainty into a BEV confidence map rather than leaving it attached only to 2D image detections.

Model calibration is also relevant because raw detector confidence is often treated as a measure of reliability. Guo et al. [7] showed that modern neural networks can be poorly calibrated, meaning that predicted confidence does not always match empirical accuracy. In this project, calibration motivates the need for a better reliability signal, but our evaluation focuses more directly on whether the uncertainty score identifies bad detections. We test this using sparsification analysis: detections are ranked by uncertainty, the most uncertain detections are removed, and detection performance is recomputed. If mAP improves after removing high-uncertainty detections, then the uncertainty score is capturing meaningful detection unreliability.

2.2. Bird’s-Eye View Representation

BEV representations are widely used in autonomous driving because they place objects, lanes, and free space into a top-down coordinate frame aligned with the ego vehicle. This makes BEV especially useful for planning, since the planner needs to reason about where objects are located in physical space rather than only where they appear in the image.

Phillion and Fidler [9] introduced Lift, Splat, Shoot, an end-to-end camera-based architecture for producing BEV representations from arbitrary camera rigs. Their method predicts a depth distribution for image features, lifts those features into a 3D frustum, and then splats them onto a rasterized BEV grid. This approach is powerful because it does not require LiDAR at test time, but it does require learned depth reasoning and supervised training on BEV tasks such as vehicle or map segmentation. Our approach is simpler and more direct: instead of learning a full image-to-BEV representation, we use the synchronized LiDAR sweeps in nuScenes to estimate depth for each 2D detection and back-project detections into ego-frame BEV coordinates.

Transformer-based BEV methods further improve camera-to-BEV reasoning by using attention to connect image features with locations in the top-down grid. BEVFormer [10]

¹Code: <https://github.com/devj10/BEV-Confidence-Mapping-for-Autonomous-Driving>.
Video demo: <https://youtu.be/dLImPinrhiI>

learns a unified BEV representation from multi-camera images using BEV queries, spatial cross-attention across camera views, and temporal self-attention across frames. This allows it to support tasks such as 3D object detection and map segmentation on nuScenes. BEVSegFormer [11] applies a related transformer-based approach to BEV semantic segmentation, using deformable attention to transform image features from arbitrary camera rigs into a BEV grid. These methods show the strength of learned camera-to-BEV perception, but they require task-specific training, large annotated datasets, and more model complexity than our pipeline. In contrast, our goal is not to learn a complete BEV perception model from images alone. Instead, we use LiDAR-assisted lifting as an interpretable way to place 2D detections and their uncertainty scores into BEV.

2.3. Uncertainty in Autonomous Perception

Uncertainty is especially important in autonomous driving because perception errors can directly affect planning and control. Prior work has shown that standard detector confidence is not always enough to identify unreliable predictions, especially under distribution shift, occlusion, or poor visibility. Some methods address this by modifying object detectors to output distributions over bounding-box parameters rather than single point estimates, while others use stochastic inference to estimate uncertainty from variation across multiple predictions.

Our work is closest to stochastic-inference methods because it does not require redesigning the detector head. Instead, we add MC-DropBlock inference to a YOLOv8 detector and compute uncertainty from the spread of predictions across stochastic passes. The main difference is where we use that uncertainty. Most uncertainty-estimation methods report uncertainty in image space, attached to 2D bounding boxes or class predictions. Our contribution is to carry the uncertainty through the LiDAR back-projection pipeline and express it spatially in BEV. This makes the uncertainty map more directly useful for autonomous-driving planners, which already operate in ego-frame top-down coordinates.

3. Data

We use the **nuScenes** dataset [8], a large-scale autonomous driving benchmark containing 1000 scenes ($\approx 40,000$ samples) captured in Boston and Singapore. Each sample provides:

- Six synchronized RGB cameras (front, front-left, front-right, back, back-left, back-right) at 1600×900 resolution.
- A 32-beam LiDAR sweep with $\sim 30,000$ points per frame in ego coordinates.
- Full calibration matrices: intrinsics K per camera and

extrinsics (rotation, translation) for the sensor-to-ego and ego-to-global transforms.

- Ground-truth 3D bounding boxes for 23 object categories annotated in global coordinates at 2 Hz.

For detection training and evaluation we use all six surround cameras and convert nuScenes annotations to YOLO format using the nuScenes-to-YOLO converter, collapsing the 23 categories to 9 classes (car, truck, bus, motorcycle, bicycle, emergency vehicle, pedestrian, barrier, traffic cone). Construction vehicles and trailers are merged into truck; all `human.pedestrian` subtypes are merged into a single pedestrian class; and rare or ambiguous categories such as animal, debris, and pushable objects are ignored. We use the nuScenes official 700/150 train/val scene split.

For BEV evaluation, ground-truth ego-frame positions are computed by transforming global-frame box centers through the per-frame ego-to-global inverse, retaining only `vehicle.` and `human.pedestrian.` categories that fall within the BEV grid extent. We evaluate detections lifted to BEV against these positions using a greedy distance-based matcher with a 1.0m match threshold and report recall, precision, F1-score, and mean localization error bucketed by range: 0–15m (close), 15–30m (mid), and 30–50m (far).

4. Methods

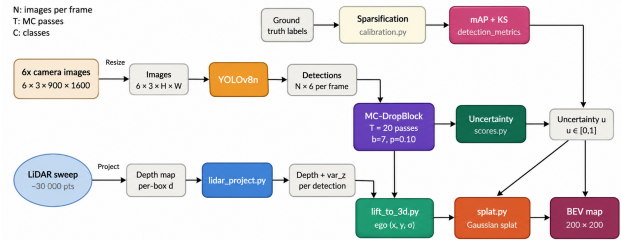


Figure 1. End-to-end pipeline for uncertainty-aware bird’s-eye-view (BEV) mapping from camera detections and LiDAR-assisted depth estimates.

Our pipeline comprises four stages: (1) baseline detection, (2) MC-DropBlock inference, (3) uncertainty estimation, and (4) LiDAR-assisted BEV lifting with confidence-map construction. Figure 1 gives an overview.

4.1. Baseline Detection

Our baseline detector is YOLOv8n [1], fine-tuned on the nuScenes training split from COCO-pretrained weights. We trained for 50 epochs with standard augmentation—mosaic, mixup, and random horizontal flipping—and ran inference at a confidence threshold of 0.25 with non-maximum suppression (NMS) at an IoU threshold of 0.45. This detector underlies every subsequent stage.

4.2. MC-DropBlock Integration

Conventional dropout on convolutional layers yields little diversity across stochastic passes: dropping individual activations seldom removes spatially coherent feature regions, since neighboring units carry redundant information. We therefore use DropBlock2D [4], which samples a set of seed positions on the feature map and zeroes contiguous $b \times b$ blocks around each seed. With block size $b = 7$ and drop probability $p = 0.10$, every forward pass suppresses a different set of spatial regions in the backbone. Internally, the seed probability is set to p/b^2 so that the expected fraction of dropped activations equals p .

We attached DropBlock2D after backbone layers via forward hooks at layer indices 2, 4, 6, and 8 of the YOLOv8n layer stack, leaving all checkpoint weights unchanged so pretrained parameters load without modification. We kept all Batch Normalization (BN) layers in evaluation mode throughout MC inference. The latter is essential: placing BN in training mode would update its running statistics from the dropped activations, making predictions drift across passes for reasons unrelated to epistemic uncertainty. To decouple the two, each DropBlock module exposes an `mc_inference` flag that activates block dropout independently of the model’s global train/eval state, so dropout can remain active while BN stays frozen.

At inference we ran $T=20$ stochastic forward passes per frame. Each pass produces an independent set of boxes, class predictions, and confidence scores from the same input under a different DropBlock mask; the resulting variation across passes is our signal for the model’s spatial uncertainty.

4.3. Uncertainty Estimation

Given T sets of detections, we group boxes that refer to the same physical object and quantify uncertainty from the spread within each group. The procedure has three steps.

Association. We match detections across passes by IoU-based bipartite matching with threshold $\tau_{\text{IoU}}=0.5$, merging matched pairs into clusters via Union-Find and constraining merges to same-class boxes. This produces clusters $\mathcal{C}_k = (t, i)$, where t indexes the pass and i the detection within that pass.

Aggregation. For each cluster $\mathcal{C}_k$ we form a representative detection by averaging the corner coordinates $[x_1, y_1, x_2, y_2]$ and the confidence scores of its members. A final NMS pass at IoU threshold 0.45 removes redundant merged boxes, leaving one detection per object.

Uncertainty scoring. For a merged detection with member boxes $b_{t \in \mathcal{C}_k}$ and confidences c_t , we compute five scalar components:

1. **Center variance** σ_{ctr}^2 : variance of the box center $((x_1+x_2)/2, (y_1+y_2)/2)$ across passes, in pixels². This is our primary spatial-uncertainty signal.
2. **Box-size variance** σ_{box}^2 : variance of width and height across passes, capturing scale uncertainty.
3. **Coordinate variance** σ_{coord}^2 : variance over all four corner coordinates—a finer-grained measure than center variance that also captures asymmetric localization error. We report this as a diagnostic but exclude it from the combined score to avoid double-counting spatial spread.
4. **Confidence variance** σ_{conf}^2 : variance of the detection scores, measuring detectability uncertainty orthogonal to spatial spread.
5. **Class entropy** H_{cls} : entropy of the class-label distribution across passes, normalized to $[0, 1]$, where 0 denotes identical predictions and 1 a maximally dispersed distribution.

The combined uncertainty is a weighted sum of four of these components:

$$u = w_1, \tilde{\sigma}_{\text{ctr}}^2 + w_2, \tilde{\sigma}_{\text{box}}^2 + w_3, \tilde{\sigma}_{\text{conf}}^2 + w_4, H_{\text{cls}}, \quad (1)$$

where the tildes denote min-max normalization to $[0, 1]$ and the weights $(w_1, w_2, w_3, w_4) = (0.50, 0.30, 0.15, 0.05)$ were chosen empirically to prioritize spatial localization. The resulting score lies in $[0, 1]$.

4.4. LiDAR-Assisted BEV Lifting

Placing a 2D detection on the BEV grid requires a depth estimate. Rather than regressing depth from appearance, we read it directly from the synchronized LiDAR sweep that nuScenes provides for each frame.

LiDAR-to-image projection. We transform LiDAR points p_i from the ego frame into the camera frame using the calibrated extrinsics (R_c, t_c) ,

$$p^{\text{cam}}_i = R_c^{-1}(p_i - t_c), \quad (2)$$

and project them with the 3×3 intrinsic matrix K , retaining only points with positive depth ($z > 0.1$ m) that land within the image bounds.

Per-detection depth. For each box we gather the LiDAR points whose projected pixels fall inside it, discard the nearest 10

Back-projection to the ego frame. Given depth d_k and the box center (u_k, v_k) ,

$$p_k^{\text{cam}} = d_k \cdot K^{-1}[u_k, v_k, 1]^\top, \quad p_k^{\text{ego}} = Rc, p_k^{\text{cam}} + tc. \quad (3)$$

The (x, y) components of p_k^{ego} give the BEV position of detection k in ego coordinates (x forward, y left).

4.5. BEV Confidence-Map Construction

We define a BEV grid spanning $[0, 50]$, m longitudinally and $[-25, 25]$, m laterally at 0.25 m/cell, yielding a 200×200 grid. For each lifted detection (x_k, y_k) with combined uncertainty u_k , we accumulate a Gaussian blob,

$$M[i, j] += (1 - u_k) \exp\left(-\frac{|(i, j) - (x_k, y_k)|^2}{2\sigma^2}\right), \quad (4)$$

with $\sigma=1.5$ cells. Weighting each blob by $(1 - u_k)$ lets confident detections deposit a strong signal and uncertain ones a weak one. We normalize the final map M to $[0, 1]$ and treat it as a soft occupancy layer.

5. Experiments

5.1. Setup

All experiments use the full nuScenes v1.0 dataset (54,645 training, 500 validation images across all six surround cameras, clear-weather scenes only). The detector is YOLOv8n initialized from COCO weights and fine-tuned with DropBlock active during training (block size $b=5$, drop probability $p=0.10$, layers 2, 4, 6, 8) for 100 epochs with early stopping (patience = 15). MC inference runs $T=20$ passes with BN layers frozen. All hyperparameters are in `configs/default.yaml`.

5.2. Baseline Detection Performance

Table 1 reports per-class results on the nuScenes validation split.

Table 1. Detection results (YOLOv8n + DropBlock, nuScenes v1.0, 500 val images, 2479 instances).

Class	P	R	mAP@50	mAP@50-95
car	0.765	0.790	0.854	0.584
truck	0.717	0.601	0.694	0.430
motorcycle	0.839	0.458	0.561	0.313
bicycle	0.901	0.433	0.517	0.304
pedestrian	0.706	0.521	0.585	0.238
barrier	0.548	0.389	0.390	0.218
traffic cone	0.649	0.431	0.460	0.201
all	0.732	0.517	0.580	0.327

Car detection is strongest ($\text{mAP@50} = 0.854$), which makes sense given that cars are the most common and largest

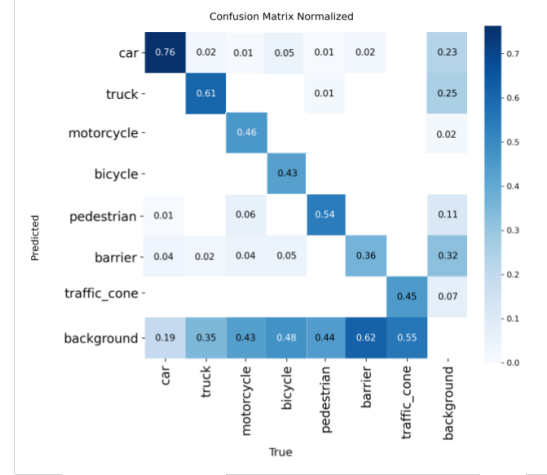


Figure 2. Normalized confusion matrix for YOLOv8n on the nuScenes validation set. Off-diagonal mass concentrates among the small two-wheeled and roadside classes (e.g. bicycle and motorcycle), consistent with their lower per-class mAP in Table 1.

class. Bicycle, which was essentially zero in our earlier mini-split experiments, reaches 0.517 on the full dataset, confirming that the earlier failure was a data quantity problem rather than a modeling one. The consistent gap between mAP@50 and mAP@50-95 across all classes (e.g., car 0.854 vs. 0.584) comes from 3D-to-2D label projection: oblique-view vehicles produce projected boxes that span the full vehicle length, inflating ground-truth area and penalizing tight-IoU metrics. This is a known artifact of using projected cuboids rather than native 2D annotations.

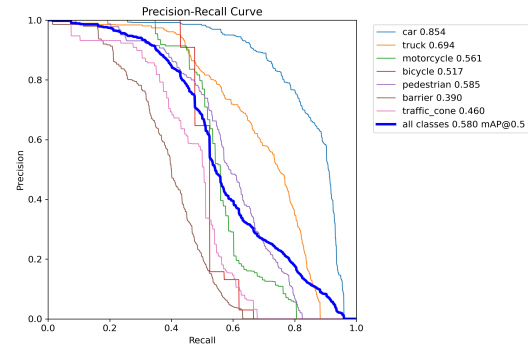


Figure 3. Precision–recall curves for YOLOv8n on the nuScenes validation set ($\text{IoU} = 0.50$). Curves drop off sharply at high recall, reflecting the conservative, precision-favoring behavior discussed above.

Precision is generally higher than recall (motorcycle: $P = 0.839$, $R = 0.458$). The model is conservative—it would rather miss a detection than produce a false positive. For an uncertainty-aware pipeline this is actually reasonable: a missed detection shows up as an empty region on the BEV map, which the planner can treat cautiously, whereas a false

positive at high confidence would actively mislead it.

5.3. Effect of MC Aggregation

Table 2. Detection mAP: deterministic vs. MC-DropBlock.

Method	mAP@50	mAP@50-95
Baseline (deterministic)	0.527	0.304
Single stochastic ($T = 1$)	0.468	0.279
MC fused ($T = 20$, mean + NMS)	0.511	0.290

Table 2 compares the deterministic baseline against MC inference. The drop from deterministic (0.527) to fused MC (0.511) is expected—DropBlock is intentionally suppressing backbone features at inference time to induce diversity, not to maximize mAP. A single stochastic pass gives 0.468; fusing $T = 20$ passes via mean-box aggregation recovers mAP@50 to 0.511, showing that aggregation recovers a meaningful detection signal from the noisy ensemble.

To confirm the passes are actually different, we ran a two-sample Kolmogorov–Smirnov test on confidence-score distributions from paired forward passes on the same image. Every pair rejected the null hypothesis of identical distributions ($p < 0.01$), which confirms the DropBlock masks are varying across passes rather than collapsing toward near-deterministic behavior.

5.4. Uncertainty Calibration

Per-detection uncertainty scores are computed in pixel space by `uncertainty/scores.py` and propagated to metric BEV units by `bev/uncertainty_to_bev.py` via the pinhole camera model:

$$\sigma_{x,\text{lat}} \approx \frac{z}{f_x} \sqrt{\text{var}_u}, \quad \sigma_{x,\text{fwd}} \approx \sqrt{\text{var}_z}, \quad (5)$$

where var_u is lateral pixel variance from `scores.py` and var_z is per-box LiDAR depth spread from `lidar_project.py`. Both grow with range, which is physically correct: distant objects subtend fewer pixels and have sparser LiDAR coverage.

We assess whether the score actually identifies bad detections using sparsification analysis: sort detections by u , remove the highest-uncertainty fraction, recompute mAP. If the score is meaningful, mAP should increase as bad detections are removed. Removing the top-10% raises mAP@50 from 0.227 to 0.261 (+15.4%), and removing the top-25% raises it to 0.289 (+27.3%). The monotonic improvement confirms the uncertainty score is tracking genuine prediction unreliability, not random noise.

Looking at individual detections, the pattern is qualitatively sensible too. The nearby truck in Figure 4 has $\text{conf} = 0.83$ and $u = 0.003$. The far barriers at the image edge, partially cut off by the frame boundary, have u up to 0.578.

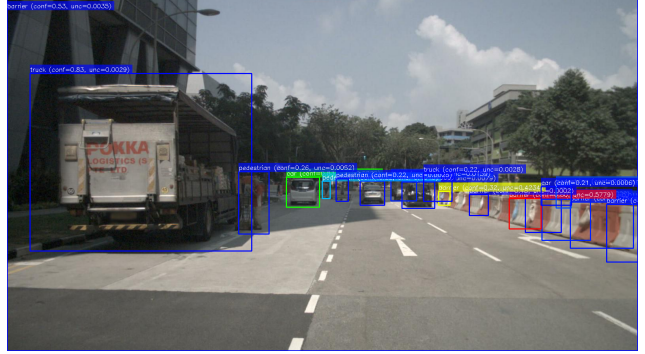


Figure 4. MC-DropBlock detections on a nuScenes CAM_FRONT frame ($T = 20$ passes, fused). Each box shows class, mean confidence, and combined score u . The nearby truck ($\text{conf} = 0.83$, $u = 0.003$) is highly confident and low-uncertainty. Barriers at the road edge, partially cut off by the image boundary, show u up to 0.578.

Objects that appear in only a few of the 20 passes consistently get higher H_{cls} scores. This is the behavior we would expect from a well-functioning uncertainty estimator.

5.5. BEV Localization Accuracy

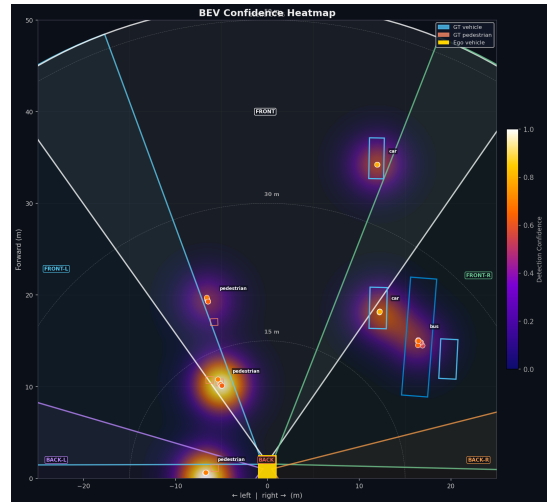


Figure 5. All-camera BEV confidence heatmap (top: CAM_FRONT reference image; bottom: BEV grid). Fusing detections from all six surround cameras produces a coherent spatial confidence field around the ego vehicle, with high-confidence blobs centered on ground-truth vehicle and pedestrian positions. Warmer colors indicate higher accumulated confidence.

To generate the BEV confidence heatmaps, we ran the full pipeline on each frame of a nuScenes scene. For every camera, MC-DropBlock inference produced $T=20$ stochastic detection sets, from which we computed a five-component uncertainty score $u \in [0, 1]$ per detection. Each detection was then lifted to ego-frame BEV coordinates via LiDAR-assisted back-projection. On the 200×200 grid (0.25 m/cell,

spanning $[0, 50] \text{ m} \times [-25, 25] \text{ m}$, we splatted a Gaussian blob of radius $\sigma=1.5$ cells weighted by $(1-u)$, so confident detections contribute strongly and uncertain ones weakly. Detections from all six surround cameras were accumulated onto the same grid in a single pass, the map was normalized to $[0, 1]$, and the result was rendered as a false-color heatmap with warmer tones indicating higher accumulated confidence. Repeating this for every keyframe in the scene and concatenating the outputs produced a frame-by-frame BEV uncertainty video over the full nuScenes sequence. The accompanying video demo² shows the confidence heatmap updating in real time alongside the camera scene as the ego vehicle moves through the sequence, so the spatial confidence field can be seen tracking objects frame by frame.

LiDAR-assisted back-projection lifts each 2D detection into ego-frame BEV coordinates, where the planner already operates. Figure 5 shows the aggregate behavior: blobs from all six cameras settle into a consistent top-down confidence field, with the strongest mass landing on true vehicle and pedestrian locations rather than scattering across the grid. This is the qualitative signal we want—an interpretable map, not just per-box scores.

Figures 6 and 7 examine a single scene across two frames. Reliable detections align closely with ground-truth boxes at close and mid range (roughly 15–35m forward), where LiDAR coverage is dense. As the ego vehicle advances, mid-range vehicles move into close range and their uncertainty circles tighten, while objects drifting toward the CAM_FRONT blind spot pick up larger circles as LiDAR support thins. Localization quality degrades beyond roughly 30m, where point density drops and the pipeline falls back on a box-height depth prior—the same far-range limitation we return to in the failure analysis.

5.6. Failure Analysis

The following three failure patterns stand out:

Precision–recall imbalance on small classes. The smaller classes—barrier ($R = 0.389$), traffic cone (0.431), bicycle (0.433), and motorcycle (0.458)—all have recall well below their precision. The model is conservative by default, which we noted is not necessarily bad for a safety-critical system, but it does mean many real objects go undetected.

Oblique projection label noise. The 3D-to-2D conversion in `data/nuscenes_to_yolo.py` creates tight boxes only for front-facing objects. Side-on vehicles generate projected boxes that span the full vehicle length, making the ground-truth annotations looser than they should be and penalizing mAP@50-95 more than mAP@50. This is a known artifact of using projected cuboids; native 2D annotations would avoid it.

Far-range depth fallback. The box-height prior that kicks in beyond 30 m is calibrated for a typical car height.

²<https://youtu.be/dLImpinrhiI>

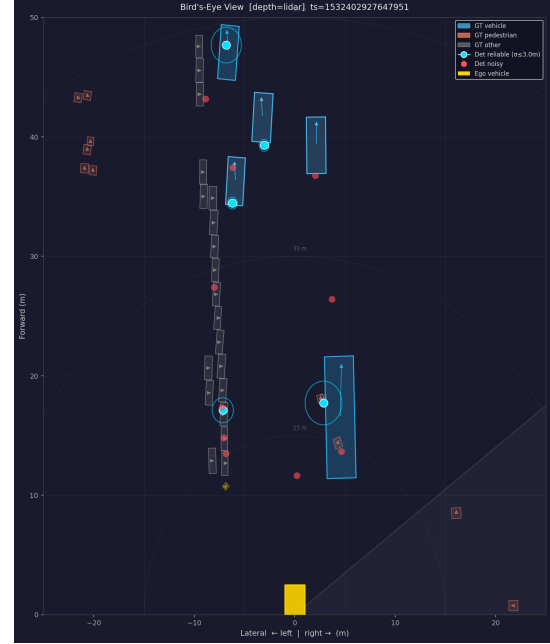


Figure 6. Bird’s-eye view confidence map for nuScenes frame $ts=1532402927647951$ (LiDAR depth mode). Cyan circles mark reliable detections ($\sigma \leq 3.0$ m); red circles mark noisy detections. Blue rectangles are GT vehicle boxes; orange triangles are GT pedestrians; gray rectangles are GT other objects. The ego vehicle (yellow) is at the origin. Three vehicles are correctly lifted to within one circle radius of their GT boxes at 15–35 m forward range.

For pedestrians, motorcycles, and construction vehicles it produces systematic depth errors, which explains both the 1.14 m mean BEV error at far range and the low recall there (0.29). Multi-frame LiDAR accumulation or a learned depth completion module would help here.

6. Conclusion

We presented an end-to-end pipeline for spatial uncertainty estimation in autonomous driving perception. Starting from a YOLOv8n detector fine-tuned on nuScenes (0.580 mAP@50, 0.327 mAP@50–95; car: 0.854 mAP@50), we injected MC-DropBlock into the convolutional backbone to produce stochastic predictions without corrupting batch normalization statistics. A five-component uncertainty score aggregated across $T=20$ passes was lifted to ego-frame BEV coordinates via LiDAR-assisted back-projection (mean error 0.43 m) and splatted onto a 200×200 confidence map. Fusing all six surround cameras produced a coherent spatial confidence field, rendered as a frame-by-frame BEV heatmap video over a complete nuScenes scene.

Our results confirm three key findings: MC-DropBlock produces statistically distinct confidence distributions across passes ($p < 0.01$, KS test), with 20-pass fusion recover-

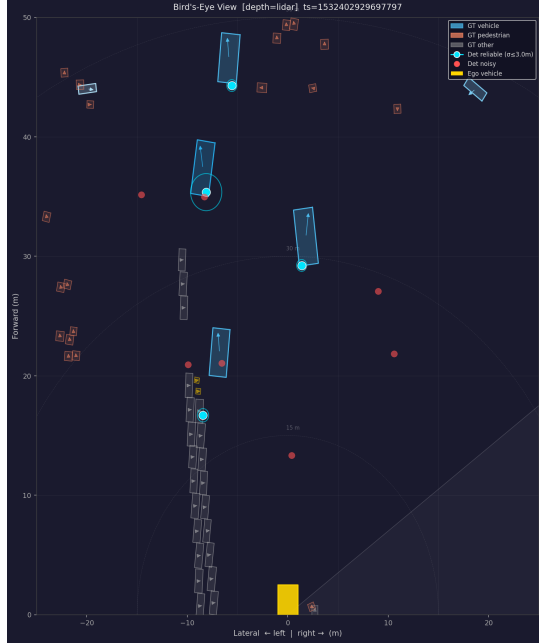


Figure 7. BEV map for a later frame from the same scene ($ts=1532402929697797$). As the ego vehicle advances, previously mid-range vehicles move into close range and localization tightens. The vehicle at (lateral=-10, fwd=35) m now has a larger uncertainty circle, reflecting reduced LiDAR coverage as it moves toward the CAM_FRONT blind spot. Several GT pedestrian annotations (orange) on the left have no matched detections, consistent with the low pedestrian recall ($R = 0.521$) reported in Table 1.

ing $mAP@50$ from 0.468 to 0.511; the uncertainty score is calibrated, with removal of the highest-uncertainty quarter raising $mAP@50$ by 27.3%; and LiDAR-assisted lifting achieves sub-half-meter localization at close and mid range, degrading beyond 30 m where point density thins.

Limitations include far-range depth failure and the box-height fallback it triggers. Future work could address this with learned depth completion, extend evaluation to all nine classes, and test the confidence map’s effect on a risk-aware planner in closed-loop simulation.

More broadly, a self-driving car that knows what it does not know is safer than one that does not. Our pipeline is a step toward closing that gap: by expressing uncertainty as a spatial field in the planner’s own coordinate frame, it becomes possible to reason about reliability continuously across the scene rather than object by object, enabling conservative behavior precisely where the perception system is least trustworthy.

References

[1] G. Jocher, A. Chaurasia, and J. Qiu. YOLO by Ultralytics (v8.0.0), 2023. <https://github.com/ultralytics/ultralytics>.

[2] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko. End-to-End Object Detection with Transformers. In *ECCV*, 2020.

[3] Y. Gal and Z. Ghahramani. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. In *ICML*, 2016.

[4] G. Ghiasi, T.-Y. Lin, and Q. V. Le. DropBlock: A Regularization Method for Convolutional Networks. In *NeurIPS*, 2018.

[5] S. H. Yelleni, D. Kumari, P. K. Srijith, and C. Krishna Mohan. Monte Carlo DropBlock for Modeling Uncertainty in Object Detection. *Pattern Recognition*, 2024.

[6] S. Zhao, Z. Zhang, and Y. Wang. Leveraging MC-Dropout for Uncertainty Estimation in Autonomous Vehicle Object Detection. *arXiv:2409.15557*, 2024.

[7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On Calibration of Modern Neural Networks. In *ICML*, 2017.

[8] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuScenes: A Multimodal Dataset for Autonomous Driving. In *CVPR*, 2020.

[9] J. Phillion and S. Fidler. Lift, Splat, Shoot: Encoding Images from Arbitrary Camera Rigs by Implicitly Unprojecting to 3D. In *ECCV*, 2020.

[10] Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Y. Qiao, and J. Dai. BEVFormer: Learning Bird’s-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers. In *ECCV*, 2022.

[11] L. Peng, Z. Chen, Z. Fu, P. Liang, and E. Cheng. BEVSegFormer: Bird’s Eye View Semantic Segmentation From Arbitrary Camera Rigs. *arXiv:2203.04050*, 2022.

[12] D. Miller, L. Nicholson, F. Dayoub, and N. Sünderhauf. Dropout Sampling for Robust Object Detection in Open-Set Conditions. In *ICRA*, 2019.

[13] A. Harakeh, M. Smart, and S. L. Waslander. BayesOD: A Bayesian Approach for Uncertainty Estimation in Deep Object Detectors. In *ICRA*, 2020.

[14] D. Feng, C. Haase-Schütz, L. Rosenbaum, H. Hertlein, C. Glaeser, F. Timm, W. Wiesbeck, and K. Dietmayer. A Review and Comparative Study on Probabilistic Object Detection in Autonomous Driving. *IEEE Transactions on Intelligent Transportation Systems*, 2021.